NATIONAL UNIVERSITY OF COLOMBIA
BOGOTÁ HEADQUARTERS - FACULTY OF ENGINEERING
DEPARTMENT OF SYSTEMS AND INDUSTRIAL ENGINEERING

Workshop 2

Presented by:

Laura Valentina Diaz Velandia
Brandon Losada Socha
Juan Daniel González Sierra
Frank Kenner Olmos Prada
Julian David Osorio Amaya

Teacher:
Carlos Andres Sierra Virguez

November 6, 2025

National University of Colombia
Faculty of Engineering
Department of Systems and Industrial Engineering
Software Engineering II
2025 - II

CRC Cards:

CRC Card: User

Class: User	
Responsibilities:	**Collaborators:**
- Store user ID, name and email	
- Encrypt and store password hash	
- Provide authentication capability	- Student, Admin

Description:

Abstract base class that represents any user in the system. Stores core user information including unique identifier (UUID), name, email, and encrypted password hash. Provides the foundational authentication capability inherited by both Student and Admin subclasses

CRC Card: Student

Class: Student	
Responsibilities:	**Collaborators:**
- Extend User with student data	- User, Program
- Store student ID and program reference	- Program
- Calculate GPA based on enrollments	- GPAService
- Calculate semester averages	- GPAService
- Manage academic progress	- Enrollment, Course
- View curriculum and course information	- CurriculumService
- Simulate course enrollment and planning	- CurriculumService, Enrollment

Description:

Represents a student user in the system. Extends User with student-specific data including student ID and a reference to their enrolled Program. Includes the calculateGPA() method to compute academic performance from course enrollments and grades. Manages academic progress and course planning.

CRC Card: Admin

Class: Admin	
Responsibilities:	**Collaborators:**
- Extend User with admin privileges	- User
- Upload and import curriculum files	- ImportService
- Validate uploaded file data	- ImportService
- Manage program and course information	- Program, Course
- Edit course credits and prerequisites	- Course

Description:

Represents an administrator user with elevated privileges. Extends User and provides administrative functionality including uploading curriculum files, editing course information (credits and prerequisites), and managing system data. Acts as the bridge between external data sources and the system.

CRC Card: Program

Class: Admin	
Responsibilities:	**Collaborators:**
- Store program ID and name	
- Organize semesters within program	- Semester
- Maintain list of all courses in program	- Course
- Provide program structure information	- Student, CurriculumService
- Calculate completion percentage	- Student

Description:

Represents an academic program offered by the university (e.g., Systems Engineering, Mechanical Engineering). Stores program identification (UUID) and name, organizes the curriculum into semesters, and maintains a complete list of all courses offered in the program. Serves as the container for the entire curriculum structure.

CRC Card: Semester

Class: Semester	
Responsibilities:	**Collaborators:**
- Store semester number and course list	- Course
- Organize courses chronologically	- Course
- Provide semester-level curriculum view	- Program, CurriculumService

Description:

Represents an academic semester within a program's curriculum. Stores the semester sequence number and maintains the list of courses scheduled for that semester. Organizes courses chronologically to reflect the natural progression of student learning paths.

CRC Card: Course

Class: Course	
Responsibilities:	**Collaborators:**
- Store course code, title, and credits	
- Store course type and prerequisites list	- Course
- Provide course details to students	- Student, CurriculumService
- Validate prerequisites for enrollment	- Enrollment
- Store in course collection for programs	- Program, Semester

Description:

Represents an individual course or subject within the curriculum. Stores course metadata including code, title, number of credits, course type (Required/Elective), weekly hours, and course description. Maintains a list of prerequisite courses and provides methods to validate prerequisites and retrieve prerequisite codes.

CRC Card: Enrollment

Class: Enrollment	
Responsibilities:	**Collaborators:**
- Associate students with courses	- Student, Course
- Record course final grade	- GPAService
- Track enrollment status	- Student
- Provide grade information for GPA calculation	- GPAService
- Store enrollment relationship	- Student

Description:

Represents the relationship between a student and a course they are taking or plan to take. Associates a specific Student with a specific Course, records the final grade achieved in the course (as a float), and tracks the enrollment status. Critical for GPA calculations and progress tracking.

CRC Card: CurriculumService

Class: CurriculumService	
Responsibilities:	**Collaborators:**
- Display complete curriculum for selected program	- Program, Course
- Recommend next courses for student	- IRecomendationStrategy, Student
- Organize curriculum by semesters	- Semester, Program
- Apply curriculum filters and display options	- Course
- Use Strategy Pattern for recommendations	- IRecomendationStrategy

Description:

Service class that orchestrates curriculum-related operations and interactions. Displays complete curriculum maps organized by semesters for selected programs, checks enrollment eligibility by validating prerequisites, simulates course enrollment to support academic planning, and provides filtered curriculum views to students.

CRC Card: GPAService

Class: GPAService	
Responsibilities:	**Collaborators:**
- Calculate student GPA	- Student, Enrollment
- Compute semester averages	- Enrollment
- Weight grades by course credits	- Course, Enrollment
- Update calculations when new grades entered	- Enrollment

Description:

Service class responsible for all GPA-related calculations. Calculates the overall GPA for a student based on their enrollments and course grades, validates that grade values are within acceptable ranges, and ensures grade integrity throughout the system. Supports decision-making in curriculum planning.

CRC Card: IRecomendationStrategy

Class: IRecomendationStrategy	
Responsibilities:	**Collaborators:**
- Define interface for recommendation algorithms	
- Enable pluggable strategy implementations	
- Support Strategy Pattern for course recommendations	

Description:

Defines the contract for different course recommendation algorithms. Supports the Strategy Pattern by allowing multiple implementations with different recommendation approaches. Enables CurriculumService to use various recommendation strategies interchangeably.

CRC Card: DefaultRecomendationStrategy

Class: DefaultRecomendationStrategy	
Responsibilities:	**Collaborators:**
- Implement IRecommendationStrategy interface	- IRecommendationStrategy
- Recommend courses following standard curriculum order	- Student, Course
- Filter courses based on completed prerequisites	- Course, Student
- Return list of recommended courses	- CurriculumService

Description:

Concrete implementation of the Strategy Pattern for course recommendations. Recommends courses following the standard, sequential curriculum order as defined in the program structure. Filters available courses based on prerequisites already completed by the student, providing straightforward course progression guidance.

CRC Card: PriorityRecomendationStrategy

Class: PriorityRecomendationStrategy	
Responsibilities:	**Collaborators:**
- Implement IRecommendationStrategy interface	- IRecommendationStrategy
- Recommend courses based on priority and difficulty	- Student, Course
- Optimize course enrollment sequence	- Student
- Return prioritized list of courses	- CurriculumService

Description:

Alternative implementation of the Strategy Pattern for course recommendations. Recommends courses based on priority levels and difficulty, allowing optimization of course enrollment sequence. Enables students to plan strategically based on course complexity and importance rather than just prerequisites.

CRC Card: ImportService

Class: ImportService	
Responsibilities:	**Collaborators:**
- Process imported curriculum files	- Admin
- Validate file format and data integrity	
- Parse data from uploaded files	
- Create Course and Program objects from import	- Course, Program
- Provide import status and error reporting	- Admin

Description:

Service class that handles importing curriculum data from external files. Processes uploaded files in CSV or JSON formats, validates file structure and data integrity, parses the imported data, and creates or updates Course and Program objects in the system based on the imported information.

CRC Card: DatabaseSingleton

Class: DatabaseSingleton	
Responsibilities:	**Collaborators:**
- Ensure single database connection instance	- DatabaseConnection
- Manage Singleton Pattern for database access	- SQLCurriculumRepository
- Provide centralized connection management	- DatabaseConnection

Description:

Implements the Singleton Pattern to ensure only one database connection instance exists throughout the application. Manages centralized database access and provides a single point of connection management for all database operations.

CRC Card: DatabaseConnection

Class: DatabaseConnection	
Responsibilities:	**Collaborators:**
- Maintain database connection	- DatabaseSingleton
- Execute queries against database	- SQLCurriculumRepository
- Retrieve and store curriculum data	- Program, Course, Student, Enrollment
- Handle connection lifecycle	

Description:

Manages the actual database connection and query execution. Maintains the connection to the database, executes queries to retrieve and store curriculum data, handles connection lifecycle (opening/closing), and serves as the data persistence layer for all domain objects.

CRC Card: SQLCurriculumRepository

Class: SQLCurriculumRepository	
Responsibilities:	**Collaborators:**
- Fetch program data	- DatabaseSingleton, Program
- Save enrollments	- DatabaseSingleton, Enrollment
- Save log entries	- DatabaseSingleton, LogEntry

Description:

Concrete implementation of the Repository Pattern for data access. Handles all database interactions related to curriculum data including fetching program information by ID, saving enrollment records, and storing audit log entries. Abstracts database operations from business logic services.

CRC Card: AuditLogService

Class: AuditLogService	
Responsibilities:	**Collaborators:**
- Record administrative actions	- Admin
- Store change details	- LogEntry

Description:

Service class that records and manages audit logs for system accountability. Records administrative actions performed by Admin users, stores details about what changes were made and when, maintains an audit trail for compliance and security purposes, and enables tracking of system modifications.

CRC Card: LogEntry

Class: LogEntry	
Responsibilities:	**Collaborators:**
- Store audit log entry	- AuditLogEntry
- Record timestamp and action	
- Track user who performed action	

Description:

Data class that represents a single audit log entry. Stores the timestamp of when an action occurred, the UUID of the user who performed the action, the type of action performed, and detailed information about the action. Provides the data structure for maintaining audit trails and compliance records.

Mockups: for a clear visualization click here
https://app.visily.ai/welcome-to-workspace/Ce1CSpeXIEoumJK8OcCe1v5fQYVkNrmp6gPVCvNwbutprxA7C9LGrObTpMFL6g3rHxCgCzR8qs4CxIqvSO5vWbcsEy7Te5hDkbKtvVQdHU1vlXghTteypQafemVlbASmcHodnAPp0Kxkcm8b0aa80c2d6c43119a6062d28626f9b4

Explanation of Purpose and Design Justification for Mockups

Mockup 1 Login/Landing Page: This landing screen's primary objective is to manage user authentication (student or administrator) and provide an entry point for unregistered users.

Relationship with requirements: It directly fulfills FR1 (The system must allow the user (student) to authenticate). It also enables guest browsing functionality, thus addressing US9 (As a Guest User, I want to browse the full curriculum of any program without logging in...).

Design justification: The centered login form and the need to protect credentials align the design with NFR3 (Security), which demands strictly restricted administrator access and protection of user credentials. The inclusion of the "Explore as Guest" option ensures immediate compliance with US9, supporting quick access to information.

Mockup 2 Program Selection: The function of this screen is to allow the student to define their academic context (Engineering program). This is a mandatory step to load the correct curricular information, considering the structure of the UNAL.

Relationship with requirements: It addresses US3 (As a Student, I want to select my engineering program so that the system shows me the correct curriculum structure). This selection ensures that the main curriculum map (Mockup 3) complies with FR2 (The system must display the complete curriculum for a selected program).

Design justification: The presentation using distinct cards, as seen in the mockup, enhances NFR2 (Usability), making the initial configuration quick and straightforward. This modular approach to selection sets the foundation for a maintainable structure, supporting NFR6 (Maintainability).

Mockup 3 Interactive Curriculum Map Principal: This is the core of the system. It is designed to provide a comprehensive and interactive visualization of the complete curriculum, organized by academic semesters (1 to 10), reflecting the student's current academic progress at a glance.

Relationship with requirements: It is the visual implementation of FR2 (display the complete curriculum organized by semesters) and FR4 (display the student's progress status). It directly satisfies US1 (As a Student, I want to view my current progress on the curriculum visually...) by using color coding (Green for Completed, Grey for Not Taken). The search bar and filter dropdowns (seen in the PDF) adhere to FR6 (allow filtering and searching) and US4 (filter by type).

Design justification: The clear organization by semester columns and the strong color coding are crucial for meeting NFR2 (Usability), ensuring the interface is intuitive. This modular view, separating the map from the progress summary, supports the Single Responsibility Principle (SRP), a cornerstone of NFR6, by making individual components responsible for distinct concerns (visualization vs. progress metrics).

Mockup 4 Subject Detail Panel / Interaction: This modal panel, activated by clicking on a subject, serves as the center for detailed information and the point of interaction for the student to manage their progress and plan future courses.

Relationship with requirements: It displays complete subject details (credits, description), satisfying FR7. The prerequisite section fulfills FR5 (display dependencies) and US6 (see a detailed prerequisite list). The visual alert (Orange, as seen in the PDF) for missing prerequisites fulfills the warning requirement set in US10. Key tracking actions implemented here include US2 (Mark as Approved) and US7 (Simulate Enrollment).

Design justification: The design, which highlights prerequisites using a warning color (Orange), provides essential Feedback to the user, enhancing usability. By encapsulating these actions in a modal, the system promotes Abstraction and adheres to Object-Oriented Design (OOD) principles, focusing the user's attention on the subject's responsibilities.

Mockup 5 Academic Report / GPA Dashboard: To consolidate and present the student's academic performance metrics in an easy-to-digest dashboard format, focusing on averages and credit distribution.

Relationship with requirements: The prominent display of the Overall GPA and Weighted GPA complies with FR11. The breakdown graph showing Completed Credits and distribution by Component (Basic, Disciplinary, Free Choice) satisfies FR4 (display student's progress status). The underlying calculations depend on the final grades input by the user via US11.

Design justification: The use of large, contrasting numbers and graphical representations (as seen in the PDF) supports the presentation of complex data, directly addressing NFR2 (Usability). By having a dedicated Report screen, the design adheres to the SRP (Single Responsibility Principle), ensuring that the complex logic for calculating averages (FR11) is separate from the curriculum visualization logic (Mockup 3).

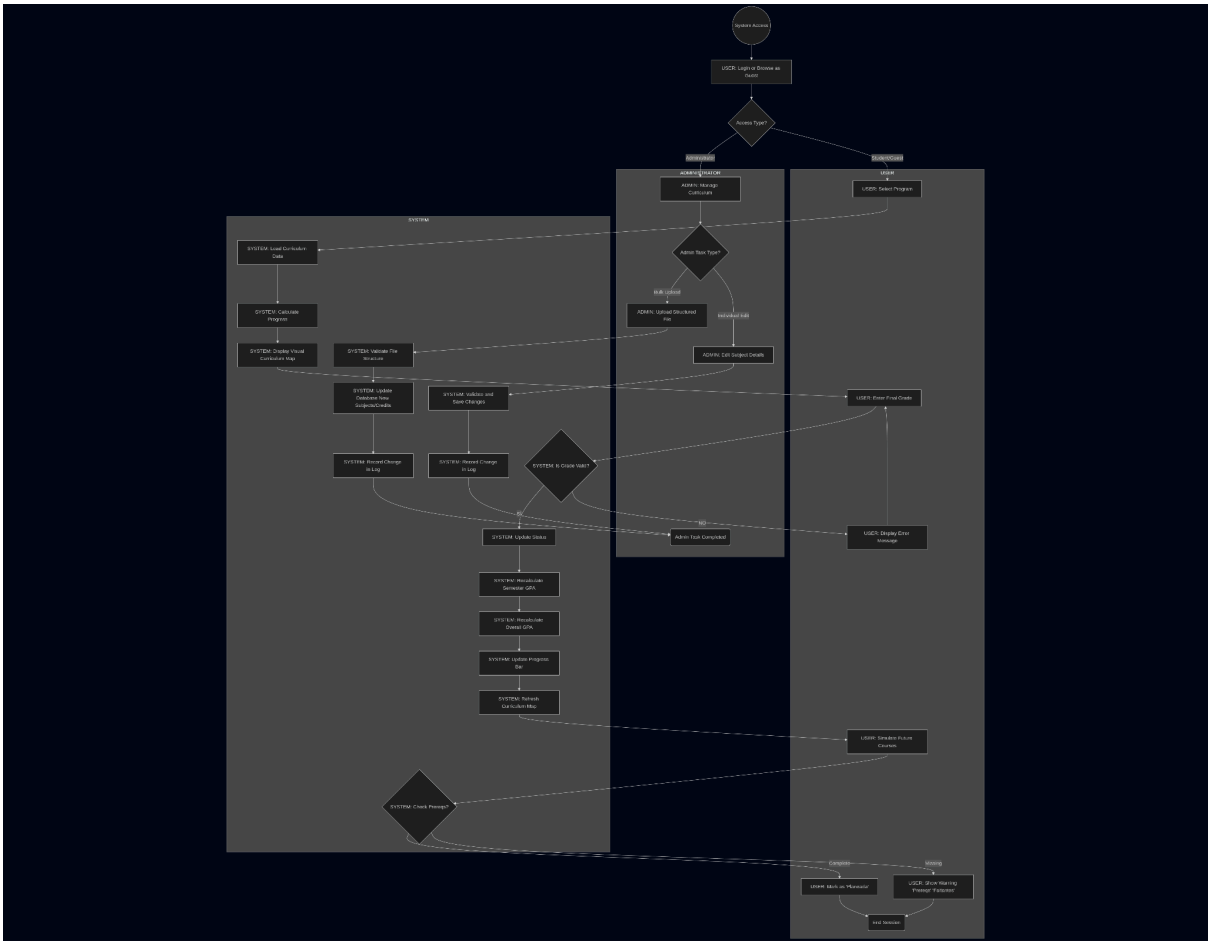
Mockup 6 Administrative Panel: To provide a restricted management interface for administrators, ensuring the maintainability and accurate update of curriculum information.

Relationship with requirements: The panel's existence and the "MODO ADMINISTRADOR" label fulfill FR8 (provide an interface for administrators) and reinforce NFR3 (Security) by requiring strictly restricted access. The bulk file upload area implements US5 (upload a new set of curriculum data via CSV/JSON), and the editing section handles US8 (edit the credit count or prerequisites for an existing subject). The "Historical Log" (seen in the PDF) satisfies the acceptance criterion for US8.

Design justification: The restricted access is paramount for NFR3 (Security). The clear separation of this module from the student interface (Mockups 3-5) is a vital application of the Separation of Concerns (SoC) principle, which is key to achieving NFR6 (Maintainability). This segregation ensures that changes in the administrative interface do not impact student functionality, allowing for independent development and deployment, which is a goal of modular design in OOD.

General Design Justification: This set of mockups establishes the formal representation of the Conceptual Design of the system, articulating what key functionalities it must execute to satisfy the Functional Requirements (FR) and User Stories (US). The justification for its design is centered on two points: Usability (NFR2), which is ensured through intuitive navigation and the consistent use of visual encoding for course statuses, and Maintainability (NFR6). Thanks to the modular structure of the screens (separating the interfaces for the Curriculum Grid, Academic Report, and Administrative Panel), this design lays the foundation for a robust implementation based on the principles of Object-Oriented Design (OOD) and SOLID. This modularity ensures that the system is open for extension, but closed for modification (OCP Principle), allowing the design to be modified and new functionalities to be added without compromising compliance with the functional and non-functional parameters already established.

Business Model Processes: for a clear visualization click here
https://acortar.link/CqzBjS



Business Model Process Description

The Business Process Model and Notation (BPMN) diagram documents the core functionality of the Interactive Curriculum Management System, focusing on the dual processes of Student Academic Tracking and Curriculum Data Maintenance. This model serves as the blueprint for the application's domain logic, ensuring alignment between user requirements and system behavior.

Process Role and Scope:

The primary role of this process is to ensure the integrity of the academic data (FR2, FR8) and to manage the Student Lifecycle within the application, from initial access to academic progress tracking (FR4, FR11).

The process begins with System Access (Mockup 1) and branches into two main flows:

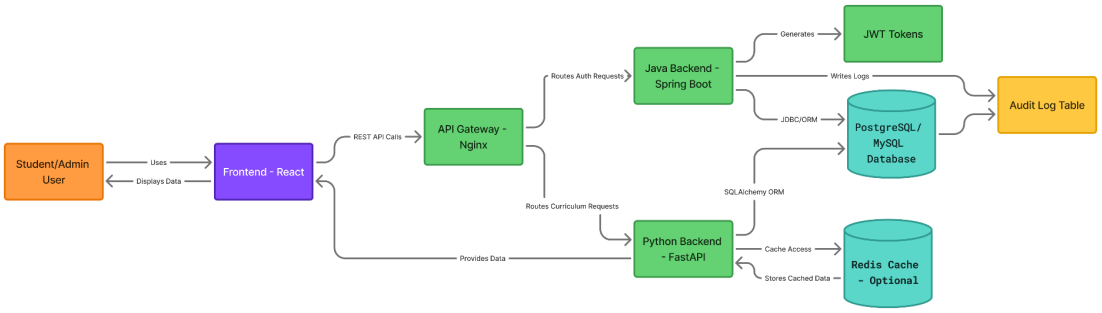
Student Flow: This sequence handles the visualization and interaction for students and guests. It covers the mandatory Select Program (US3) leading to the Display Visual Curriculum Map (US1, Mockup 3). Crucially, this flow documents the complex sequence of entering grades (Enter Final Grade), including the essential Error Handling Loop (ValidateGrade -- NO --> ShowError -> EnterGrade), ensuring data quality before calculating and updating the GPA (FR11). It ends with the planning and simulation features (US7).

Administrator Flow: This segregated lane, accessed via the AccessType -- Administrator --> ManageCurriculum gateway, justifies the Admin Panel (Mockup 6). It documents the procedures for maintaining the curriculum, including bulk data loading (US5) and individual subject editing (US8), fulfilling FR8. The separation of this flow into its own partition demonstrates a clear Separation of Concerns (SoC), which is a foundational element for achieving Maintainability (NFR6) in the software architecture.

By detailing the sequence of steps and relationships between the user and the system components, this BPMN clarifies the problem and guides the solution, ensuring that the software development team understands the expected flow before moving to the Technical Design phase.

Architecture Diagram:

Given the course requirements, which implementations requires the use of REST APIs, Java and Python as programming languages for the backend and Relational Databases (SQL) for the database services; and in addition, the functional requirements for the application defined in workshop 1, the following architecture diagram was designed:



For better visualization, consider accessing: https://www.figma.com/board/OKO4PJW3gOW4bIi74wPHSJ/Interactive-Student-Curriculum-Architecture-Diagram?node-id=0-1&t=yLnG0WpiCJSN5tIr-1

Explanations:

Architecture follows a multi-tier and service-oriented pattern, it divides the system in three principal layers:

- **Presentation Layer (Frontend):** React-based interface that handles the visualization, user input and communication with the APIs defined in the backend.
- **Application Layer (Backend): Two services:**
  - A Java (Spring Boot framework) service that handles authentication, authorization and administrative control over the application.
  - A Python (FastAPI framework) service that executes the, so considered, "business logic" (curriculum management, GPA computation for registered users, recommendations).
- **Data Layer (PostgreSQL/MySQL):** Central relational store whose principal function is to maintain consistency and support SQL queries for course and student data.

This multi-tier pattern can allow the application to be maintainable and scalable, aligning with SOLID principles. Also, there are some additional features, like the optional caching table which can be useful for user data retrieval speed.

Class Diagram:

As for the architecture diagram, the global requirements for the application pushed the following class UML diagram:

For better visualization, consider accessing:
https://acortar.link/PkPfTu

Class Diagram Explanation

The UML Class Diagram was designed under the guidelines of Object-Oriented Design (OOD) and prioritizing the SOLID Principles. The structure distinguishes three main categories of components: Domain Entities, Business Logic Services, and Infrastructure/Abstraction Components.

Domain Entities: These classes model the core elements of the system, including User, Student, and Admin (which inherit from User), and the academic structure (Program, Semester, Course). The Enrollment class manages the many-to-many relationship between Student and Course, storing the final grade (FR3, US11).

Business Logic Services: These classes encapsulate complex functional responsibilities, adhering to the Single Responsibility Principle (SRP):

- CurriculumService: Responsible for displaying the map (Mockup 3), planning (US7), and validating prerequisites (US10). It collaborates with IRecommendationStrategy for course suggestions.
- GPAService: Isolates the responsibility of the complex GPA calculation (FR11, Mockup 5) and incorporates grade validation (validateGrade), a requirement of the BPMN Error Handling Loop (seen in the BPMN).
- ImportService: Exclusively handles the logic for processing and validating external files (US5), collaborating with Admin (Mockup 6).

Infrastructure and Abstraction Components (SOLID): These components reinforce Maintainability (NFR6):

- ICurriculumRepository: A key abstraction interface that implements the Dependency Inversion Principle (DIP). It separates the Business Logic (Services) from database details. The high-level services depend on this interface, not the concrete implementation.
- SQLCurriculumRepository: Is the concrete implementation of the ICurriculumRepository interface, coupled to the persistence system (DatabaseSingleton).
- AuditLogService & LogEntry: These classes manage the logging of changes. Their existence is fundamental to meet the Historical Log section of Mockup 6 and the acceptance criterion of US8.

Key Class Diagram Relationships

The diagram's relationships are defined to reflect the academic structure and, crucially, the dependencies of the services based on advanced design principles.

A. Generalization and Structural Relationships:

- **Inheritance (extends):**
  - Student and Admin inherit from User (role generalization, Mockup 1).
- **Composition (*--, mandatory relationships):**
  - Semester is composed of Course.
  - Student is composed of Enrollment.
- **Aggregation (o--, part-whole relationships):**
  - Program aggregates Semester.
  - Student aggregates Program.

B. Collaboration and SOLID Principles:

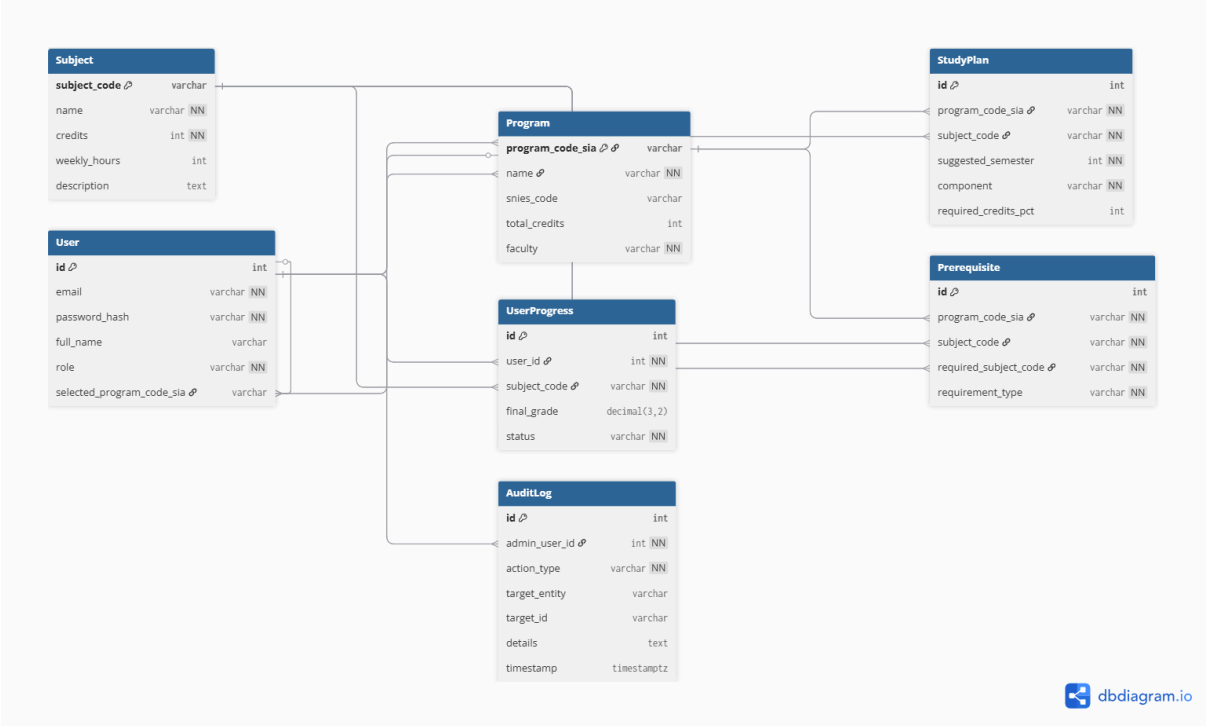
- **Dependency Inversion (DIP):**

    - CurriculumService, GPAService, and ImportService depend on the ICurriculumRepository abstraction. This ensures that business logic is not coupled to the database implementation (e.g., SQLCurriculumRepository).
- **Strategy Pattern:**
    - CurriculumService uses the IRecommendationStrategy interface and its concrete implementations (DefaultRecommendationStrategy, PriorityRecommendationStrategy).
- **Single Responsibility (SRP) and Auditing:**
    - Admin uses (uses) AuditLogService. This dependency is essential to log curriculum modifications (US8) and maintain the history of changes (Mockup 6) without overloading the Admin class with log persistence logic.
    - ImportService also uses AuditLogService to log bulk data loading (US5).
- **Functional Dependency:**
    - GPAService uses (uses) validation methods, such as validateGrade (which supports the validation loop in the BPMN), to ensure data quality before proceeding to the final GPA calculation (FR11).

Design Patterns:

1. **Strategy Pattern:**
   Implemented via the IRecommendationStrategy interface and its variants (Default…, Priority…). It allows the curriculum recommendation logic to be easily changed.
2. **Singleton Pattern:**
   Implemented via DatabaseSingleton, guarantees a single shared database connection across services (caching database isn't taken into account). It centralizes resource management.

Relational Database Model:



[Relational Database Model Interactive Student Curriculum]

Github repository:

https://github.com/josorioam404/Interactive-Student-Curriculum